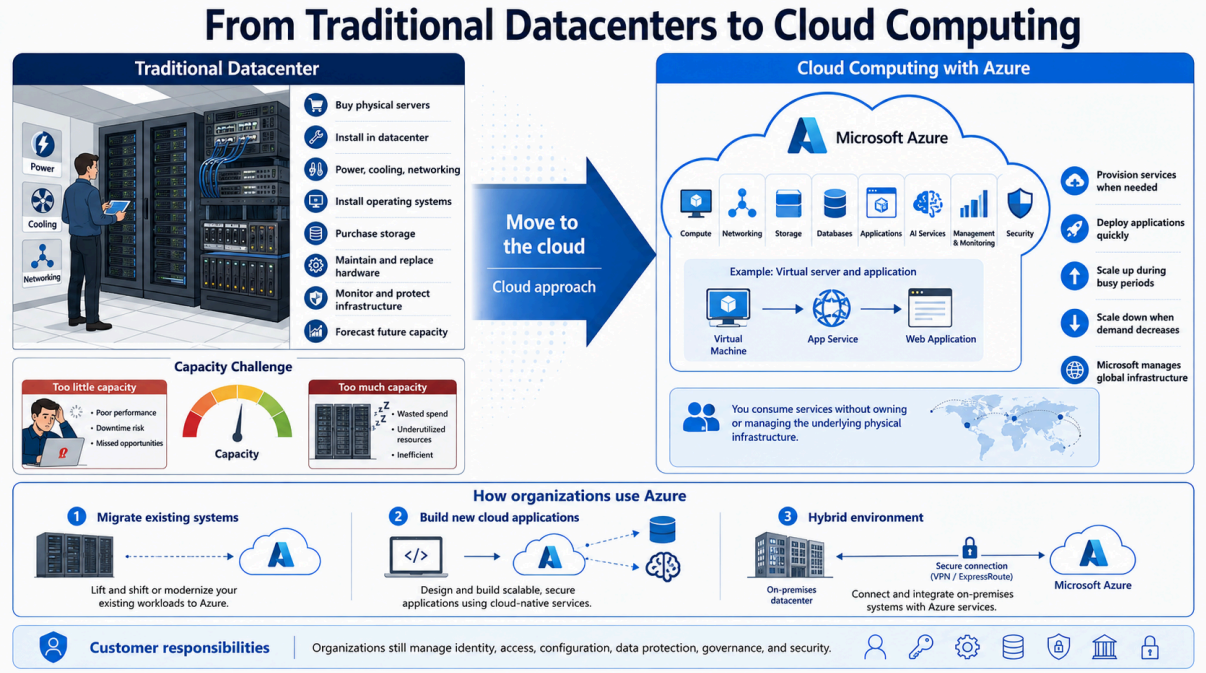
Manage identity, access, and governance

Azure Foundations for Cloud Security



This diagram explains the shift from a traditional datacenter model to cloud computing with Microsoft Azure, and why that shift matters when we talk about security. On the left side, we see the traditional datacenter approach. In this model, an organization is responsible for buying physical servers, installing them in a datacenter, providing power, cooling, and networking, installing operating systems, purchasing storage, maintaining hardware, monitoring the environment, and planning future capacity. In other words, the organization is responsible not only for running applications, but also for managing the underlying physical infrastructure that supports those applications.

The image also highlights one of the biggest operational challenges in a traditional datacenter: capacity planning. If an organization deploys too little capacity, the result can be poor performance, downtime, and missed business opportunities. If it deploys too much capacity, the organization wastes money on underutilized hardware and resources. This makes traditional infrastructure expensive and difficult to scale efficiently, especially when business demand changes over time.

The right side of the diagram shows the cloud computing model with Microsoft Azure. Instead of purchasing and managing physical infrastructure, organizations consume cloud services as needed. Azure provides a broad set of services such as compute, networking, storage, databases, applications, AI services, management and monitoring, and security capabilities. The key benefit is that these services can be provisioned quickly and adjusted based on demand. For example, organizations can deploy applications faster, scale up during busy periods, and scale down when demand decreases. Microsoft manages the global physical infrastructure, which removes much of the burden associated with owning and maintaining datacenter hardware.

The diagram also shows that organizations can use Azure in different ways. One option is to migrate existing systems from an on-premises datacenter into Azure. Another option is to build new cloud-native applications directly in Azure by using modern platform services. A third option is a hybrid environment, where on-premises systems remain in place but are securely connected to Azure using technologies such as VPN or ExpressRoute. This is important because many organizations do not move everything to the cloud at once. Instead, they often operate in a mixed environment where some systems remain on-premises while others run in Azure.

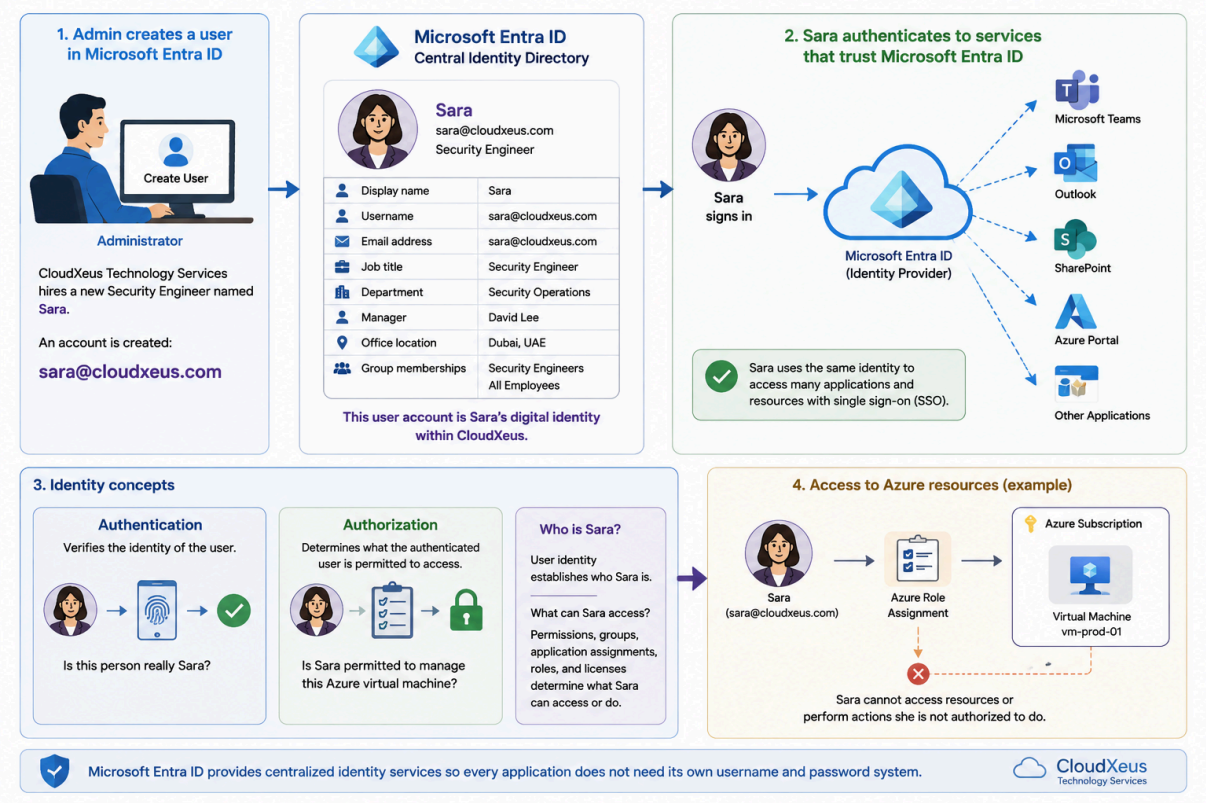
From a security perspective, the most important message in this diagram is that moving to the cloud does not remove the organization’s security responsibilities. Even though Microsoft manages the underlying physical infrastructure, the customer still has important responsibilities. The diagram highlights that organizations continue to manage areas such as identity, access, configuration, data protection, governance, and security. This means cloud security is not just about trusting Microsoft to secure everything. Instead, it

requires the organization to correctly configure and protect its own resources, identities, workloads, and data.

This becomes the foundation for everything we study in cloud security. Before implementing controls such as role-based access control, Conditional Access, Key Vault, Defender for Cloud, or network protection, we first need to understand the environment in which those controls operate. Azure changes how infrastructure is delivered, but it does not eliminate the need for secure design and secure administration. In fact, because cloud resources can be deployed quickly and at scale, mistakes in configuration can also spread quickly if they are not properly managed.

The key takeaway from this diagram is that Azure gives organizations flexibility, scalability, and faster service delivery, but security remains a shared effort. Microsoft secures the cloud infrastructure, while the customer remains responsible for securing identities, data, applications, configurations, and access to those services. Understanding that balance is essential because it forms the starting point for building secure Azure and hybrid environments.

About Microsoft Entra ID



This diagram explains the role of **Microsoft Entra ID** as the central identity and access management service used across Microsoft cloud environments. The easiest way to understand Microsoft Entra ID is to think of it as the system that stores digital identities and then helps determine how those identities authenticate and what they are allowed to access.

The example begins with CloudXeus Technology Services hiring a new Security Engineer named **Sara**. An administrator creates a user account for Sara in Microsoft Entra ID. The account includes information such as her display name, username, email address, job title, department, manager, office location, and group memberships. Together, these properties form Sara's digital identity within the organization.

Once the account exists, Sara can use that identity to sign in to services that trust Microsoft Entra ID. For example, the same identity can be used to access Microsoft Teams, Outlook, SharePoint, the Azure portal, and many other applications. This is one of the major benefits of centralized identity management. Individual applications do not need to maintain completely separate username and password systems because they can rely on Microsoft Entra ID as the identity provider.

This also introduces the concept of **single sign-on**, or SSO. Sara can authenticate using her organizational identity and then access multiple applications that trust Microsoft Entra ID. Depending on the organization's security policies, additional controls such as multifactor authentication or Conditional Access can also be applied when she signs in.

The lower part of the diagram introduces two concepts that are extremely important throughout cloud security: **authentication and authorization**. Authentication answers the question, “Who are you?” When Sara signs in, Microsoft Entra ID verifies that the person attempting to access the environment really is Sara. This could involve a password, Microsoft Authenticator, a passkey, or another supported authentication method.

Authorization happens after authentication and answers a different question: “What are you allowed to do?” Successfully proving that Sara is who she claims to be does not automatically give her access to every resource in the organization. Her permissions, group

memberships, application assignments, Azure roles, and other access controls determine what she can actually access.

The diagram therefore separates two important ideas. Sara's **identity** establishes who she is, while her **permissions and assignments** determine what she can do. This distinction is critical because an authenticated user can still be denied access to a resource when the necessary authorization has not been granted.

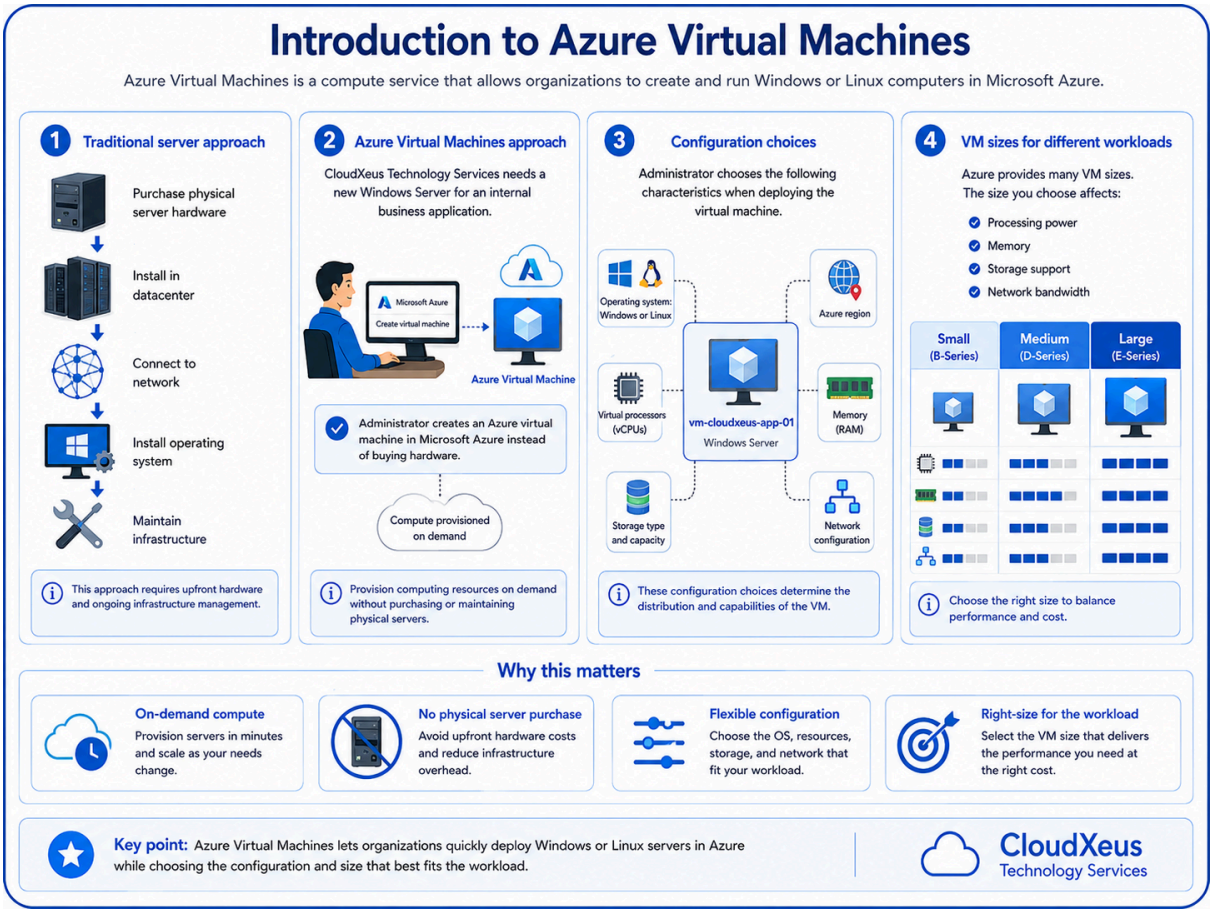
The Azure example on the right demonstrates this. Sara may successfully sign in with her Microsoft Entra ID account, but if she needs to manage a virtual machine such as **vm-prod-01**, she must also have an appropriate Azure role assignment. For example, an administrator might assign a role that allows her to manage that virtual machine at the required scope. Without the necessary role assignment, authentication can succeed while access to the Azure resource is still denied.

This introduces the principle of **least privilege**. Users should only receive the permissions they actually need to perform their responsibilities. Sara's job as a Security Engineer does not mean that she should automatically have unrestricted access to every Azure subscription, virtual machine, application, or piece of organizational data. Access should be deliberately assigned based on her role and business requirements.

Microsoft Entra ID therefore becomes a central component of cloud security. It allows organizations to centrally create and manage identities, authenticate users, integrate applications, manage group memberships, and connect identities to authorization systems such as Azure role-based access control.

The key takeaway from this diagram is that **identity is the starting point for access**. Microsoft Entra ID establishes and authenticates the digital identity, while authorization controls determine what that identity can access or perform. Understanding the difference between identity, authentication, and authorization is essential because these concepts appear repeatedly when we work with Azure resources, Microsoft 365, enterprise applications, privileged access, and many other security controls.

Introduction to Azure Virtual Machines



This diagram introduces **Azure Virtual Machines**, or Azure VMs, and explains how organizations can use them to run Windows or Linux servers in Microsoft Azure without purchasing and maintaining physical server hardware.

The diagram begins by comparing Azure Virtual Machines with the traditional server approach. In a traditional datacenter, an organization first needs to purchase physical server hardware. That server then needs to be installed in a datacenter, connected to the network, configured with an operating system, and maintained throughout its lifecycle. This means the organization is responsible not only for the workload running on the server, but also for the physical infrastructure supporting it.

With Azure Virtual Machines, the approach is different. Imagine that CloudXeus Technology Services needs a new Windows Server for an internal business application. Instead of purchasing another physical server, an administrator can create a virtual machine directly in Microsoft Azure. Azure provides the underlying physical

infrastructure, while the organization provisions the compute resource it needs on demand.

This is one of the main advantages of cloud computing. A virtual machine can normally be provisioned much faster than a traditional physical server. The organization does not need to wait for hardware to be purchased, delivered, installed, and connected before the operating system can be used.

When creating an Azure virtual machine, however, the administrator still needs to make several important configuration decisions. One of the first choices is the **operating system**. Azure supports both Windows and Linux virtual machines, so the operating system selected should match the requirements of the application or workload that will run on the machine.

The administrator must also choose the **Azure region** where the virtual machine will be deployed. The region determines the geographic location in which the Azure resources supporting the virtual machine are hosted. Organizations may choose regions based on factors such as proximity to users, regulatory requirements, service availability, resiliency design, and application architecture.

The next important consideration is the amount of **compute power** required. Azure represents processor capacity using virtual CPUs, commonly referred to as vCPUs. A lightweight application may require only a small amount of processing power, while a more demanding workload may require considerably more.

The amount of **memory**, or RAM, must also be selected. Different applications have different memory requirements, so the VM configuration needs to provide enough resources for the operating system and application to run effectively.

Storage is another important component. Azure virtual machines use virtual disks for the operating system and application data. Administrators need to consider the type and capacity of storage required based on factors such as performance, workload characteristics, and cost.

The virtual machine also needs a **network configuration**. Azure networking determines how the VM communicates with other Azure resources, on-premises systems, the Internet, or other networks. Later in the course, this becomes particularly important from a

security perspective because network access to virtual machines must be carefully controlled.

All of these choices come together to create a virtual machine such as the example shown in the diagram, **vm-cloudxeus-app-01**. The important point is that the administrator is not simply creating a generic server. They are selecting the operating system, processing power, memory, storage, networking, and deployment location that best match the requirements of the workload.

Azure provides many different **VM sizes** for this reason. A VM size defines the amount and combination of resources available to the virtual machine. Different VM families are optimized for different workload requirements. Some are designed for general-purpose workloads, while others provide greater CPU, memory, storage, or other capabilities.

The diagram uses small, medium, and large examples to show the basic idea. A smaller VM may be suitable for a lightweight application or development environment. A larger VM may be needed for an application that requires more processing power, memory, storage performance, or network bandwidth.

Choosing the correct VM size is important because it affects both **performance and cost**. Selecting a VM that is too small can result in poor application performance. Selecting a VM that is much larger than necessary can result in unnecessary cloud spending. Administrators therefore need to choose a size that provides the resources required by the workload without significantly overprovisioning it.

The lower part of the diagram summarizes why Azure Virtual Machines are useful. First, they provide **on-demand compute**. Organizations can provision servers when they need them instead of purchasing hardware in advance. They can also change the resources assigned to a workload as its requirements evolve.

Second, organizations do not need to purchase and maintain the physical server itself. Microsoft operates and maintains the underlying Azure datacenter infrastructure.

Third, Azure VMs provide **flexible configuration**. Administrators can select the operating system, compute resources, storage, networking, and other options that best suit the workload.

Finally, organizations can **right-size virtual machines** based on their requirements. This allows them to balance application performance

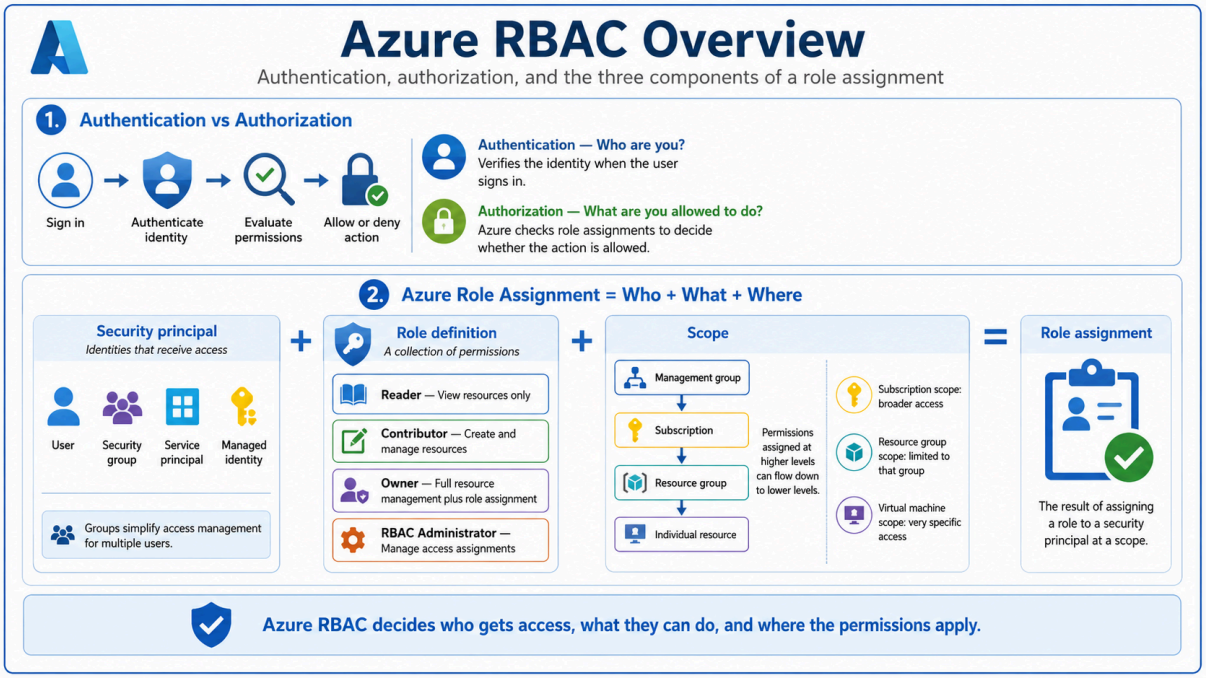
with cloud cost.

From a security perspective, it is also important to understand that Azure managing the physical infrastructure does not mean Microsoft manages everything inside the virtual machine. The customer is still responsible for many aspects of the VM, including operating system configuration, identities, permissions, network access, applications, data, updates, and security settings.

This distinction becomes very important as we move further into Azure security. A virtual machine may be easy to deploy, but it still needs to be properly protected. We need to consider who can administer the VM, how users connect to it, which network traffic is permitted, how secrets are protected, whether the operating system is patched, how disks are secured, and how threats are detected.

The key takeaway from this diagram is that **Azure Virtual Machines provide on-demand Windows or Linux servers without requiring organizations to purchase physical hardware**. Administrators can choose the configuration and size that best match the workload, while still remaining responsible for securing and managing the operating system, applications, identities, data, and access to the virtual machine.

What is Azure Role-Based Access Control



This diagram explains **Azure Role-Based Access Control**, commonly referred to as **Azure RBAC**, and how it is used to control access to Azure resources. The simplest way to understand Azure RBAC is that it helps answer three questions: **Who is getting access, what are they allowed to do, and where do those permissions apply?**

The diagram first separates **authentication** from **authorization**. Authentication answers the question, “Who are you?” When a user signs in, Microsoft Entra ID verifies the identity of that user. Once the identity has been authenticated, Azure then evaluates what that identity is allowed to do.

That second step is authorization. Authorization answers the question, “What are you allowed to do?” Azure RBAC uses role assignments to determine whether a user or another identity is permitted to perform an action on an Azure resource.

For example, a user may successfully sign in to the Azure portal, but that does not automatically mean they can create, modify, or delete resources. Azure checks the permissions assigned to that identity before allowing the requested operation.

The diagram then introduces the three main components of an Azure role assignment: **security principal, role definition, and scope**. A useful way to remember this is **Who + What + Where**.

The first component is the **security principal**. This represents the identity that receives access. A security principal could be an

individual user, a security group, a service principal, or a managed identity. Instead of assigning permissions separately to many users, organizations can often assign roles to groups and then manage membership of those groups.

The second component is the **role definition**. The role definition describes what actions the security principal is allowed to perform. Azure includes many built-in roles designed for common administrative and operational tasks.

For example, the **Reader** role allows an identity to view resources without making changes. The **Contributor** role allows an identity to create and manage resources, but does not normally allow that identity to manage role assignments. The **Owner** role provides broad resource management permissions and can also manage access. Azure also provides roles specifically designed for managing RBAC access assignments.

The important point is that a role is essentially a collection of permissions. Instead of assigning individual permissions one by one, administrators assign a role that contains the actions needed for a particular job.

The third component is the **scope**. Scope determines where the permissions apply. Azure provides a hierarchy of scopes that can include a management group, subscription, resource group, or an individual resource.

For example, imagine that an administrator assigns Sara the Reader role at the subscription level. Sara can then view resources throughout that subscription, subject to the permissions provided by the role.

If the same Reader role is assigned only at the resource group level, Sara's permissions are limited to that resource group and the resources contained within it.

If the role is assigned directly to a single virtual machine, the access is much more specific and applies only to that virtual machine.

This hierarchy is important because permissions assigned at a higher scope can normally flow down to lower scopes. For example, a role assignment at the subscription level can apply to resource groups and resources within that subscription. Administrators should therefore carefully choose the scope of every role assignment.

When the security principal, role definition, and scope are combined, the result is a **role assignment**. In simple terms, the role assignment tells Azure that a particular identity receives a particular set of permissions at a particular location in the Azure resource hierarchy.
For example, an administrator might assign the **Virtual Machine Contributor** role to a support team at the scope of a specific resource group. This would allow members of that team to manage virtual machines in that resource group without automatically giving them administrative access across the entire subscription.
This is where the principle of **least privilege** becomes extremely important. Organizations should grant only the permissions required to perform a job and should apply those permissions at the smallest practical scope.
If a user only needs to view resources, assigning the Owner role would provide far more access than necessary. Similarly, if someone only needs to manage a single resource group, assigning permissions across the entire subscription would create unnecessary security exposure.
Azure RBAC is therefore one of the primary mechanisms used to enforce authorization across Azure. It helps organizations separate responsibilities between administrators, developers, security teams, applications, and automated workloads while controlling exactly what each identity is permitted to do.
The key takeaway from this diagram is that **Azure RBAC controls who gets access, what actions they can perform, and where those permissions apply**. Authentication establishes the identity, while Azure RBAC handles authorization. A role assignment combines a security principal, a role definition, and a scope, allowing organizations to implement structured access control and follow the principle of least privilege across their Azure environments.

Lab - RBAC - Resource level assignment

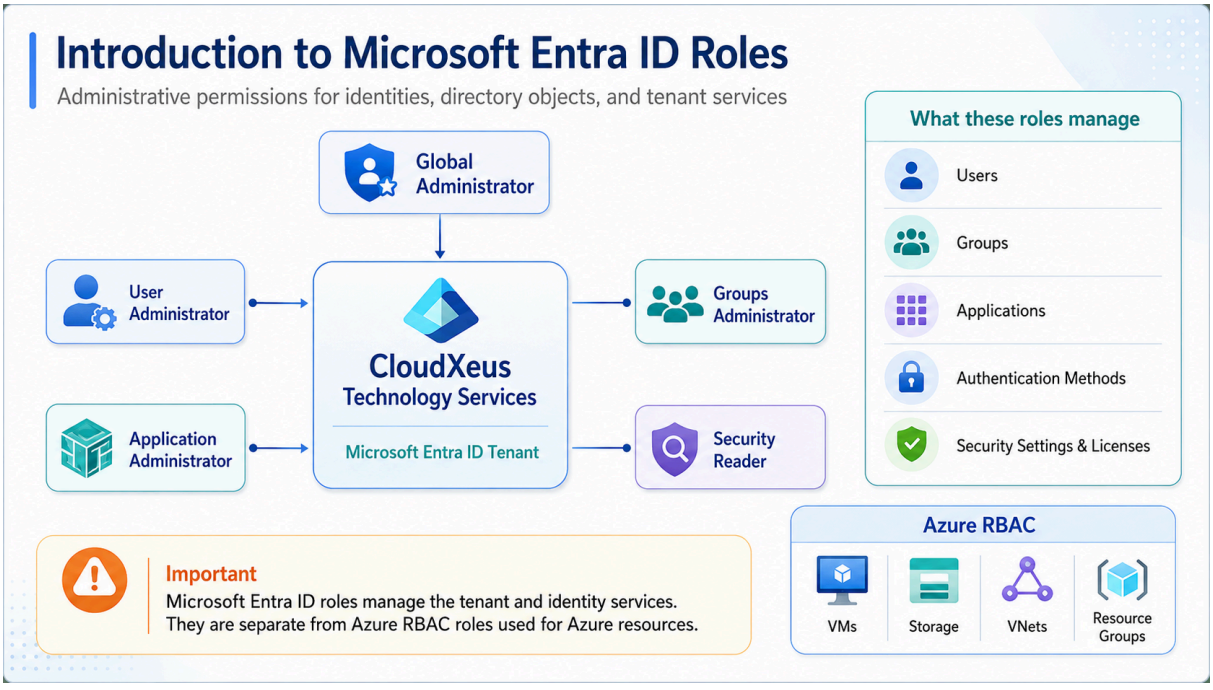
https://learn.microsoft.com/en-us/azure/role-based-access-control/built-in-roles

Creating a Custom Azure RBAC Role

Role permissions

Microsoft.Storage/*/read*
Microsoft.Network//read
Microsoft.Compute/*/read*
Microsoft.Compute/virtualMachines/start/action
Microsoft.Compute/virtualMachines/restart/action
Microsoft.Authorization//read
Microsoft.ResourceHealth/availabilityStatuses/read
Microsoft.Resources/subscriptions/resourceGroups/read

Introduction to Microsoft Entra ID Roles



This diagram introduces **Microsoft Entra ID roles** and explains how they are used to delegate administrative responsibilities within a Microsoft Entra tenant.
The central idea is that Microsoft Entra ID roles are designed to control **administrative access to identity and directory services**.

They determine who can manage users, groups, enterprise applications, authentication methods, security-related settings, licenses, and other tenant-level capabilities.

In the example shown, CloudXeus Technology Services has a Microsoft Entra ID tenant. Different administrators are given different roles depending on the tasks they need to perform. This allows the organization to divide responsibilities instead of giving every administrator unrestricted control.

At the top of the diagram is the **Global Administrator** role. This is one of the most powerful Microsoft Entra roles because it provides broad administrative access across the tenant. A Global Administrator can manage many aspects of the directory and can also assign other Microsoft Entra roles.

Because this role provides such extensive access, it should be assigned to as few users as possible. Organizations should avoid using Global Administrator for routine administration when a more specific role can perform the required task.

The diagram then shows several examples of more specialized roles. A **User Administrator** is intended for tasks related to managing users and certain aspects of groups. For example, an administrator responsible for employee accounts may need to create users, update account properties, or perform supported password-management tasks. That user does not necessarily need full administrative control over applications or security settings.

A **Groups Administrator** focuses on group management. Groups are commonly used to organize users, assign access, distribute licenses, and simplify permission management. Assigning a dedicated role for group administration allows someone to perform these tasks without receiving unrelated tenant-wide permissions.

The **Application Administrator** role is intended for managing application-related objects in Microsoft Entra ID. This becomes important when organizations register applications, configure enterprise applications, manage service principals, or integrate applications with Microsoft Entra ID for authentication and authorization.

The diagram also includes the **Security Reader** role. This is a good example of a read-focused administrative role. A Security Reader can review supported security-related information without receiving the

same level of administrative capability as a role that can modify security settings.

These examples demonstrate an important security principle: **administrative responsibilities should be separated based on job requirements**. If an employee only needs to manage users, there is usually no reason to give that person Global Administrator permissions.

This is another application of the **principle of least privilege**. Administrators should receive only the permissions they need to perform their assigned responsibilities and no more.

Microsoft Entra ID includes many built-in roles because different administrative tasks require different levels of access. Some roles focus on identities, while others focus on applications, authentication, security, devices, licensing, or specific Microsoft services.

One of the most important distinctions shown in the diagram is the difference between **Microsoft Entra ID roles and Azure RBAC roles**. Microsoft Entra ID roles are used primarily to manage the **tenant and identity services**. For example, they can provide administrative permissions over users, groups, applications, authentication methods, and directory settings.

Azure RBAC, on the other hand, is used to control access to **Azure resources** such as virtual machines, storage accounts, virtual networks, resource groups, and subscriptions.

This distinction can sometimes be confusing because both systems use the word “role,” but they control different things.

For example, imagine that Sara is assigned the User Administrator role in Microsoft Entra ID. That role may allow her to perform user administration tasks in the tenant. It does not automatically give her permission to manage an Azure virtual machine.

Similarly, assigning Sara the Virtual Machine Contributor role through Azure RBAC may allow her to manage virtual machines at a particular Azure scope, but it does not automatically allow her to create Microsoft Entra users or modify authentication methods.

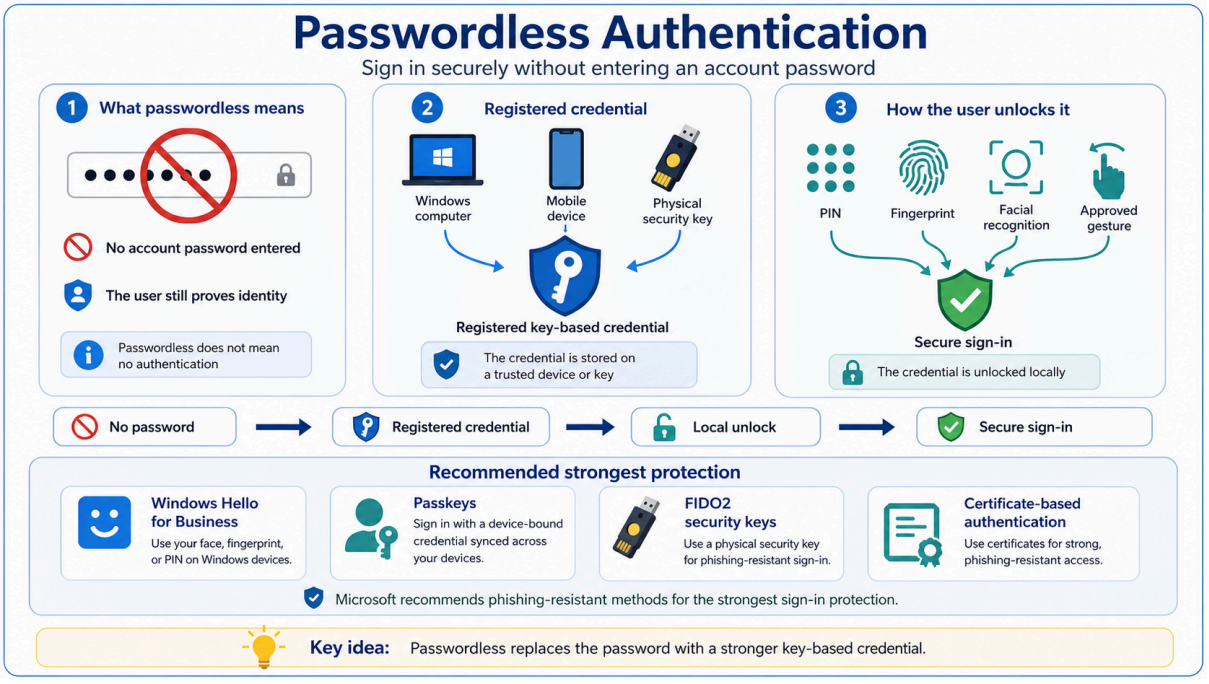
It is therefore useful to think about the two systems separately. **Microsoft Entra ID roles answer the question: What administrative tasks can this identity perform within the identity and directory environment?**

Azure RBAC answers the question: What actions can this identity perform against Azure resources, and at what scope?
In real organizations, a person may need permissions from both systems. For example, a security administrator could have a Microsoft Entra role that allows them to review identity security information and an Azure RBAC role that allows them to review resources within a particular Azure subscription.
The key takeaway from this diagram is that **Microsoft Entra ID roles provide administrative permissions over identities, directory objects, and tenant services, while Azure RBAC controls access to Azure resources**. Understanding this distinction is essential because both role systems are heavily used in Microsoft cloud security, but they solve different authorization requirements.

Introduction to Microsoft Entra Authentication Methods

https://learn.microsoft.com/en-us/entra/identity/authentication/overview-authentication

Moving Beyond Passwords with Microsoft Entra Passwordless Authentication



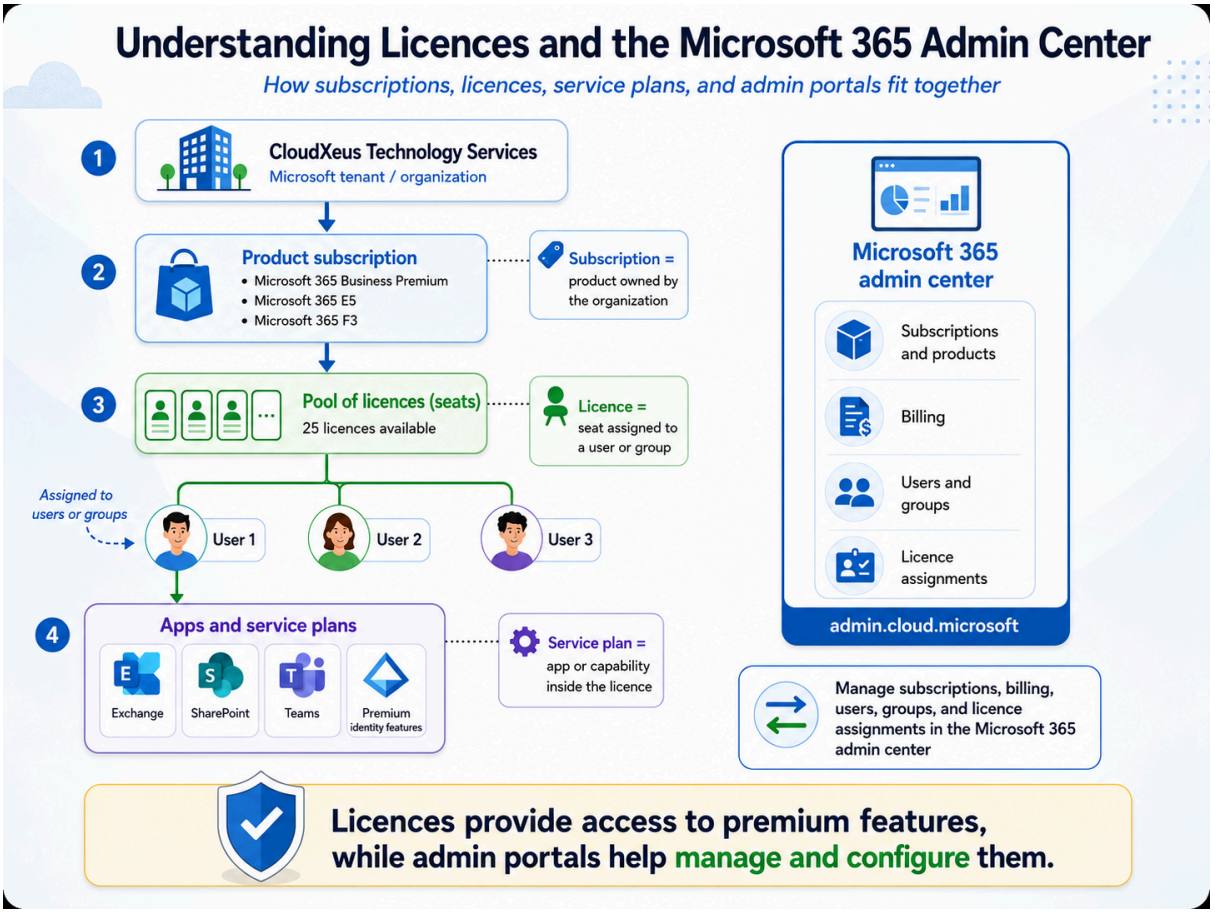
💡 **Passwordless authentication** allows users to securely sign in without entering their traditional account password. The user still has to prove their identity, but instead of relying on a reusable password, the authentication process uses a stronger registered credential associated with a trusted device or security key.

Examples include **Windows Hello for Business, passkeys, and FIDO2 security keys**. The user can unlock the credential locally using a PIN, fingerprint, facial recognition, or another supported gesture. The important point is that the PIN or biometric is used to unlock the credential—it is not simply sent across the network as a replacement password.

This approach provides stronger protection because passwords can be guessed, reused, stolen, or captured through phishing. Passwordless methods use cryptographic credentials, and phishing-resistant methods such as FIDO2 security keys and suitable passkeys can significantly reduce the risk of credential theft.

The key takeaway is that **passwordless does not mean authentication disappears—it means the password is replaced by a stronger method of proving identity**. For sensitive and privileged accounts, organizations should prioritize strong, phishing-resistant authentication wherever practical.

Understanding Microsoft Cloud Licences



Microsoft cloud licensing can be understood as a hierarchy of **subscriptions, licences, and service plans**. An organization such as CloudXeus Technology Services purchases a Microsoft product subscription, such as Microsoft 365 E5, and that subscription provides a pool of licence seats that can be assigned to users.

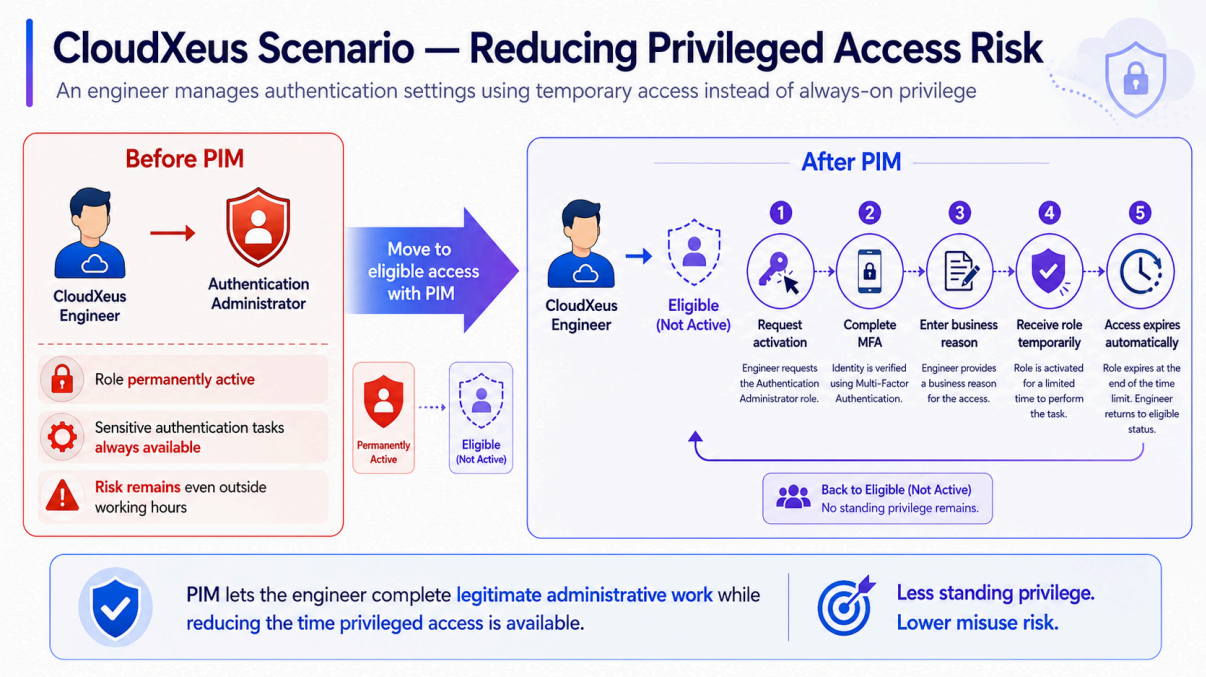
A **licence** is the entitlement assigned to a user, while **service plans** are the individual services and capabilities contained within that licence. For example, a Microsoft 365 licence could provide access to Exchange, SharePoint, Teams, and premium identity or security features depending on the product purchased.

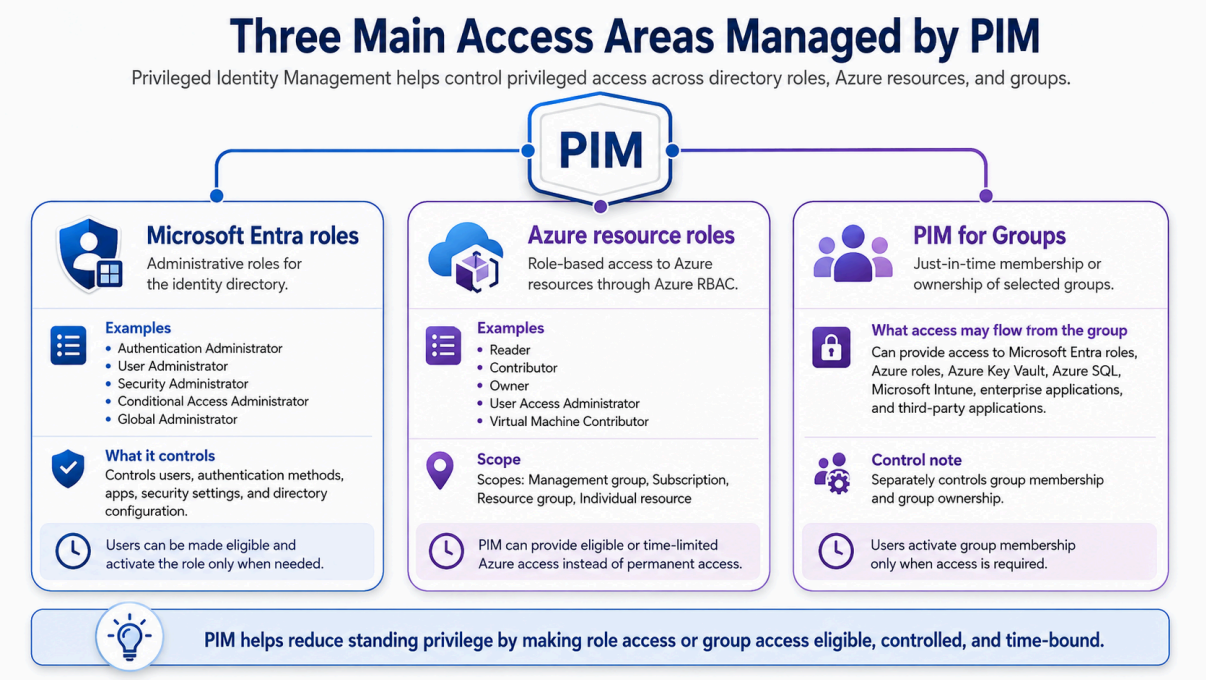
Administrators commonly use the **Microsoft 365 admin center** to view subscriptions, billing information, users, groups, and licence assignments. Licensing and administration are different concepts: a licence makes a feature available, while the appropriate administration portal is used to configure that feature.

The key takeaway is: **subscription = product purchased by the organization, licence = seat assigned to a user, and service plan = capability contained within that licence**. This matters in security

scenarios because advanced identity and security features may require specific Microsoft licensing before they can be used.

Privileged Identity Management





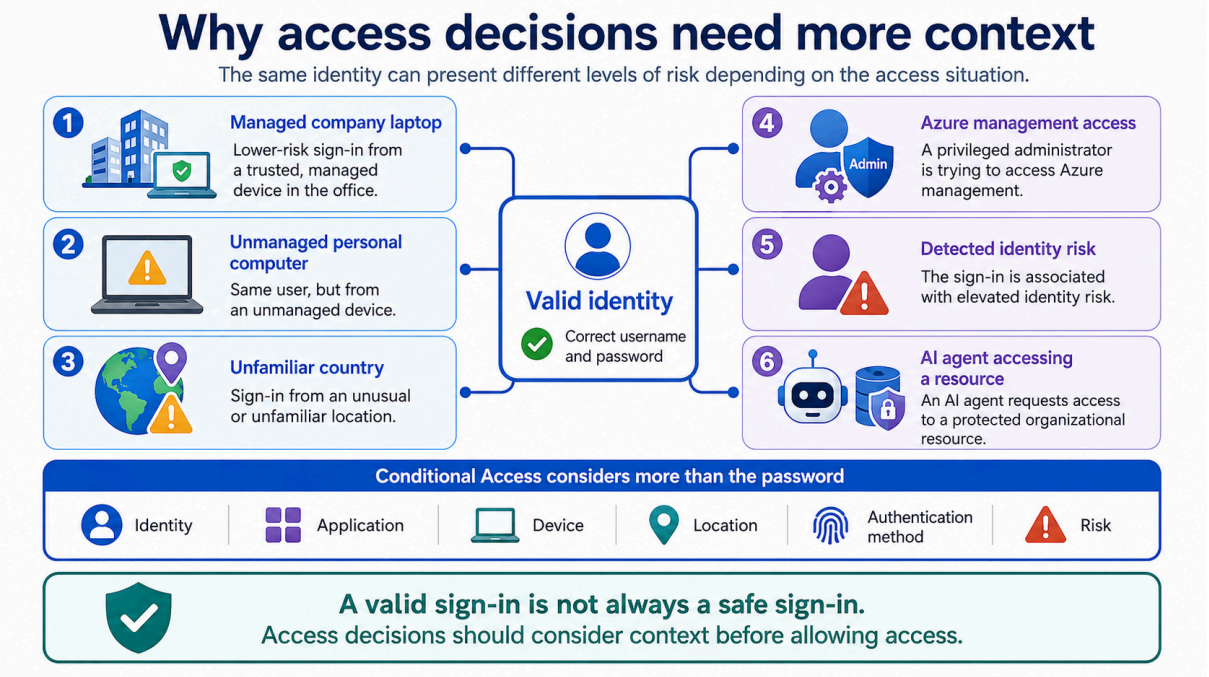
💡 **Microsoft Entra Privileged Identity Management**, or PIM, helps reduce the risk associated with permanently assigned administrative privileges. Instead of leaving a sensitive role such as Authentication Administrator active all the time, an administrator can be made **eligible** for the role and activate it only when privileged work needs to be performed.

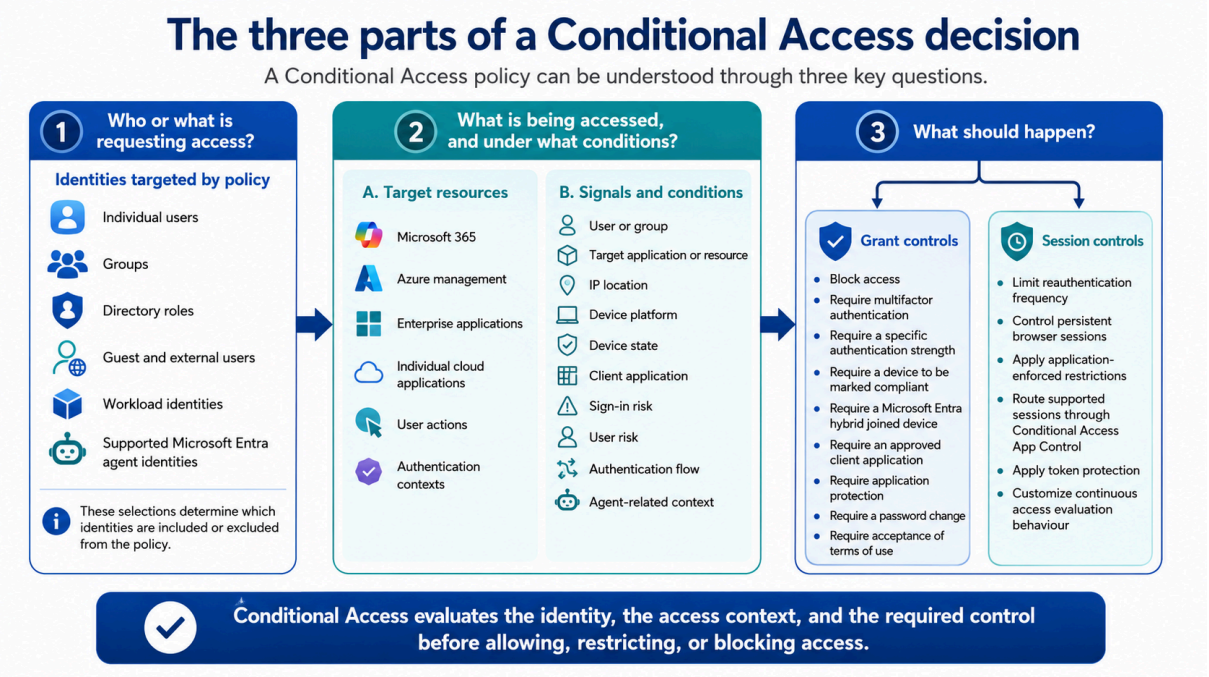
During activation, organizations can require controls such as **multifactor authentication, business justification, approval, and a limited activation period**. The administrator receives the role temporarily, performs the required task, and then the role expires so that the user returns to an eligible but inactive state.

PIM can help govern several forms of privileged access, including **Microsoft Entra roles, Azure resource roles, and eligible membership or ownership through PIM for Groups**. This means the same just-in-time principle can be applied to directory administration, Azure RBAC permissions, and access that is inherited through sensitive groups.

The key takeaway is that PIM reduces **standing privilege**. Instead of **permanently privileged**, the preferred model becomes **eligible → activate when required → perform the task → access expires**. This supports least privilege and reduces the time during which powerful permissions can potentially be misused.

Microsoft Entra Conditional Access





Microsoft Entra Conditional Access helps organizations make access decisions using more than just a valid username and password. The same user can present very different levels of risk depending on the device being used, location, application being accessed,

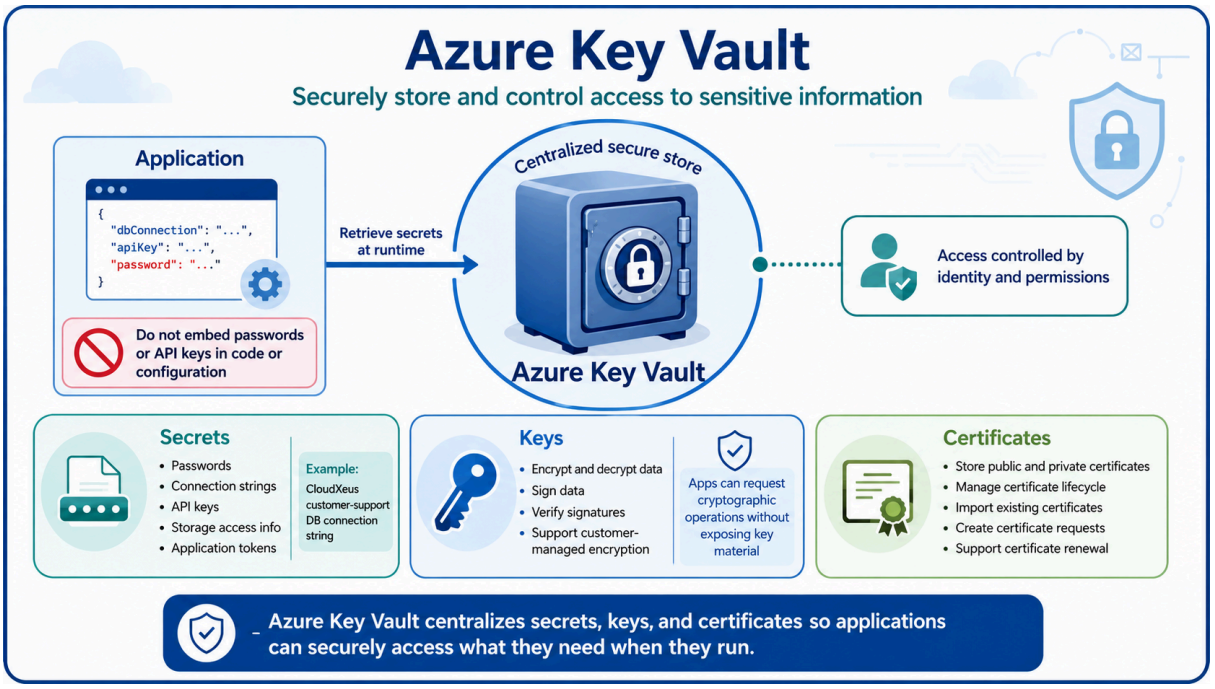
authentication method, detected risk, or whether privileged administrative access is being requested.

A Conditional Access policy can be understood through three questions: **Who or what is requesting access? What resource is being accessed and under what conditions? What should happen?** Policies can target users, groups, roles, workload identities, and supported agent identity scenarios, while evaluating signals such as device state, location, authentication, and risk.

Based on those conditions, Microsoft Entra can apply **grant controls** such as blocking access, requiring MFA, requiring a specific authentication strength, or requiring a compliant device. **Session controls** can also influence what happens after access is granted, such as requiring reauthentication or applying restrictions to the resulting session.

The key takeaway is that **a valid sign-in is not automatically a safe sign-in**. Conditional Access evaluates the identity and the surrounding context before deciding whether access should be allowed, restricted, or blocked. A useful formula is: **Identity + Resource + Conditions → Required access control**.

Introduction to Azure Key Vault

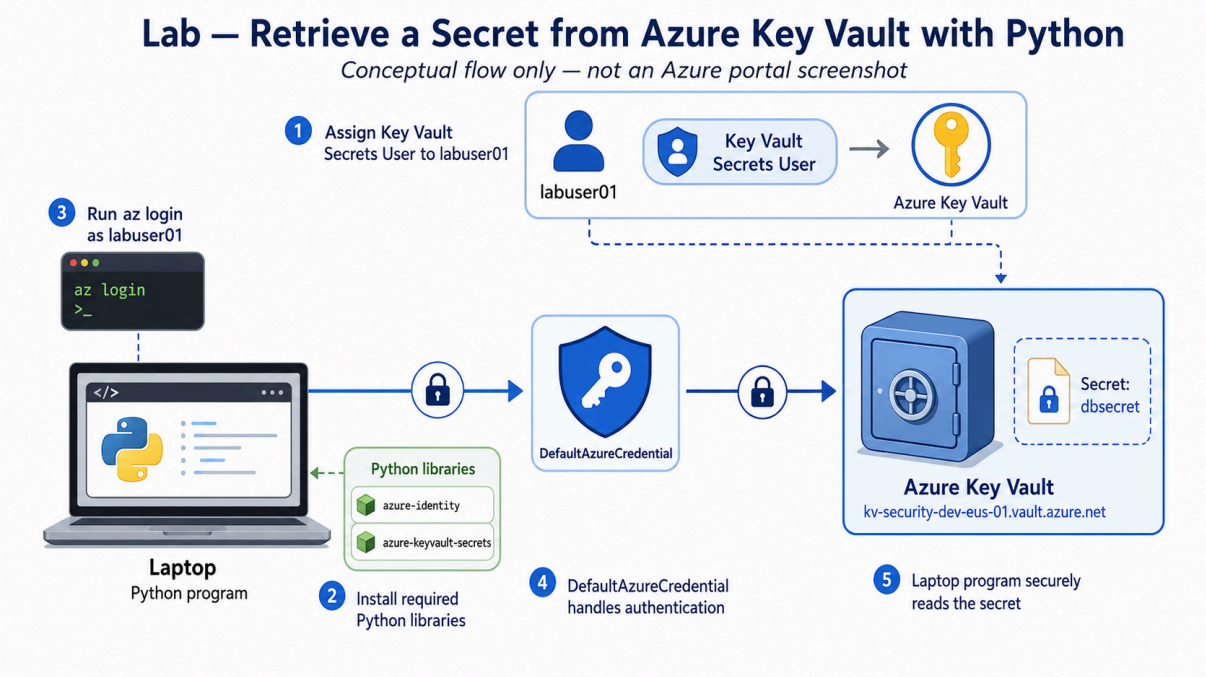


Azure Key Vault provides a centralized location for securely storing and controlling access to sensitive information used by applications. Instead of embedding passwords, connection strings, API keys, or other credentials directly inside source code or configuration files, applications can retrieve them securely from Key Vault when needed. Key Vault primarily works with three types of objects: **secrets, keys, and certificates**. Secrets can contain items such as passwords and connection strings. Keys are used for cryptographic operations such as encryption, decryption, signing, and key wrapping. Certificates can be stored and managed for scenarios such as TLS and application authentication.

Access to this sensitive information is controlled through **identity and authorization**. For example, an application can use a managed identity to authenticate with Microsoft Entra ID and then receive only the Key Vault permissions it requires. An application that only needs to read a secret should not automatically receive permission to manage keys or certificates.

The key takeaway is to **separate sensitive information from application code**. Store secrets, keys, and certificates centrally in Key Vault, authenticate applications using Microsoft Entra identities, and apply least-privilege permissions so applications can retrieve or use only what they genuinely need.

Lab – Retrieve a Secret from Azure Key Vault with Python



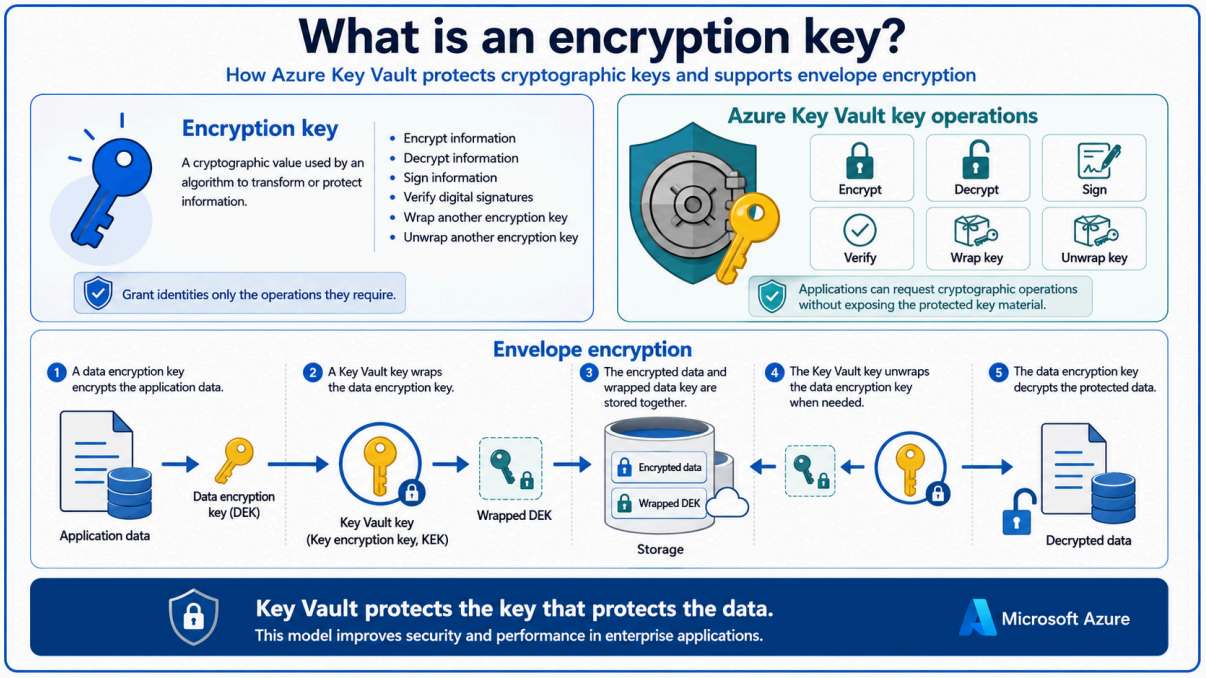
In this lab, a Python application running on a laptop retrieves a secret called **dbsecret** from Azure Key Vault. The user `labuser01` is first assigned the **Key Vault Secrets User** role so that the identity has permission to read secret values without receiving unnecessary administrative access to the vault.

The laptop then uses the `azure-identity` and `azure-keyvault-secrets` Python libraries. The user signs in with `az login`, which establishes an authenticated Microsoft Entra session. The Python application uses **DefaultAzureCredential**, which can use that Azure CLI authentication context during local development.

When the program requests `dbsecret`, two different security checks occur. **Authentication** establishes that the request is being made as `labuser01`, while **authorization** checks whether that identity has the required Key Vault permissions. A successful sign-in alone does not provide secret access—the appropriate role assignment must also exist.

The key takeaway is the security pattern: **authenticate using an identity, authorize that identity with least privilege, and retrieve the secret dynamically rather than storing it in the application**. In production, the same general application design can use a managed identity instead of relying on an interactive `az login`.

Quick look at creating encryption keys



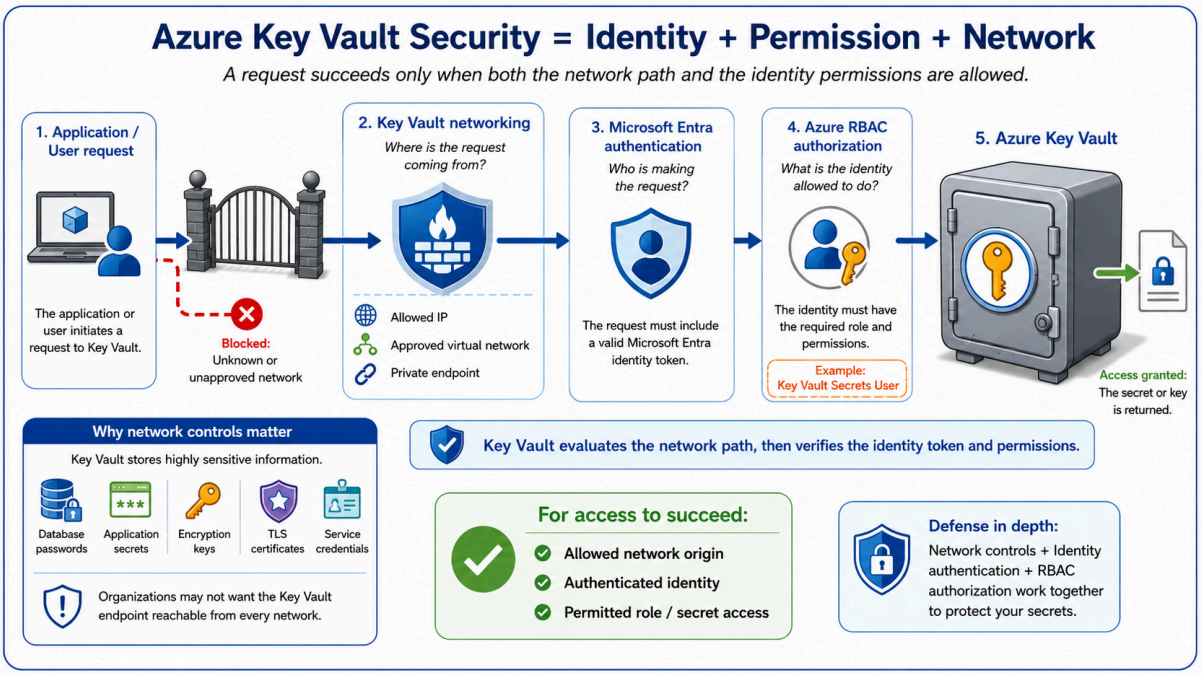
An **encryption key** is a cryptographic value used to protect information. Azure Key Vault can securely store cryptographic keys and allow authorized applications to perform operations such as **encrypt, decrypt, sign, verify, wrap key, and unwrap key** without unnecessarily exposing the protected key material.

An important principle is to grant identities only the key operations they actually require. For example, an application that only needs to wrap and unwrap another encryption key should not automatically receive permissions to perform every possible cryptographic or administrative operation.

The diagram also introduces **envelope encryption**. A **Data Encryption Key (DEK)** encrypts the actual application data. A separate **Key Encryption Key (KEK)** stored in Key Vault then protects, or wraps, the DEK. The encrypted data and wrapped DEK can be stored together because the wrapped DEK cannot be used until an authorized operation unwraps it.

The easiest relationship to remember is: **the DEK protects the data, and the KEK protects the DEK**. This approach provides efficient encryption for large amounts of data while keepin

Configure Azure Key Vault Firewall and Network Access



Azure Key Vault security should be viewed as a combination of **network access, authentication, and authorization**. An identity might have permission to read a secret, but the request can still be blocked if it originates from a network that the Key Vault firewall does not allow.

Key Vault networking can be restricted using mechanisms such as **approved public IP addresses, selected virtual networks, service endpoints, and private endpoints**. These controls help organizations reduce the number of networks from which sensitive secrets, keys, and certificates can be reached.

After the network path is permitted, Microsoft Entra ID authenticates the caller and establishes **who is making the request**. Azure RBAC then determines **what that identity is allowed to do**. For example, an authenticated managed identity with the Key Vault Secrets User role may be allowed to retrieve secrets, while another identity without that role is denied.

The key takeaway is that these controls provide **defense in depth**. For Key Vault access to succeed, the request must come through an allowed network path, use a valid Microsoft Entra identity, and have the required permissions. Remember: **network controls determine**

where access can come from; identity and RBAC determine who can access what.

Move the Key Vault Application to an Azure Virtual Machine

The following commands can be used as a reference for this chapter

1. Update the package list

Refreshes the Ubuntu package index so the virtual machine knows about the latest available packages and versions before we install anything.

```
sudo apt update
```

2. Install Python and pip

Installs Python 3 along with `pip`, which we will use to install the Azure SDK libraries required by our application.

```
sudo apt install -y python3-full python3-pip
```

3. Create the project folder

Creates a dedicated folder for the Key Vault application and then moves us into that directory so all project files remain together.

```
mkdir keyvault-lab
cd keyvault-lab
```

4. Create a Python virtual environment

Creates an isolated Python environment for the application. This keeps the libraries used by this lab separate from the system-wide Python installation.

```
python3 -m venv .venv
```

5. Activate the virtual environment

Activates the virtual environment. Any Python packages we install from this point are installed into this project environment.

```
source .venv/bin/activate
```

6. Install the Azure Python libraries

Installs the Azure SDK libraries needed by the application. `azure-identity` handles Microsoft Entra authentication, while `azure-keyvault-secrets` allows Python to work with Key Vault secrets.

```
python -m pip install azure-identity azure-keyvault-secrets
```

7. Install the Azure CLI

Installs the Azure CLI on the virtual machine. We use it initially to authenticate to Azure and verify that the VM can communicate with our Azure environment.

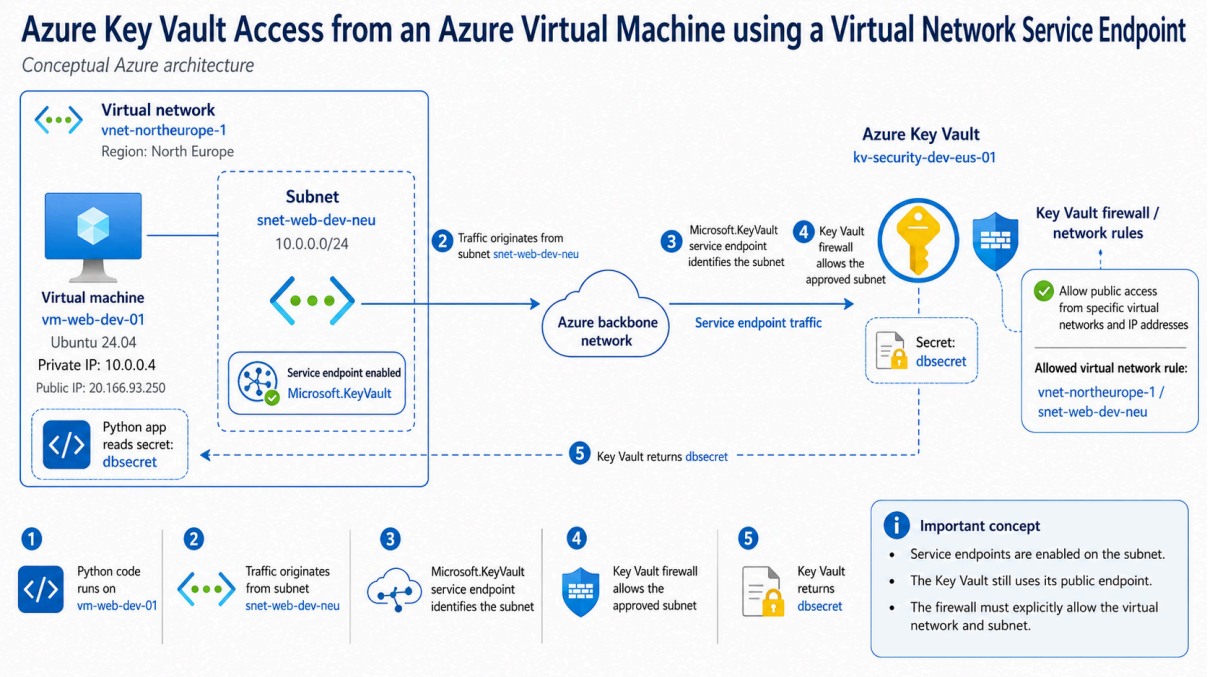
```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

8. Sign in to Azure

Starts an interactive Microsoft Entra sign-in using a device code. We open the provided sign-in page on another device, enter the code, and authenticate with the required Azure account.

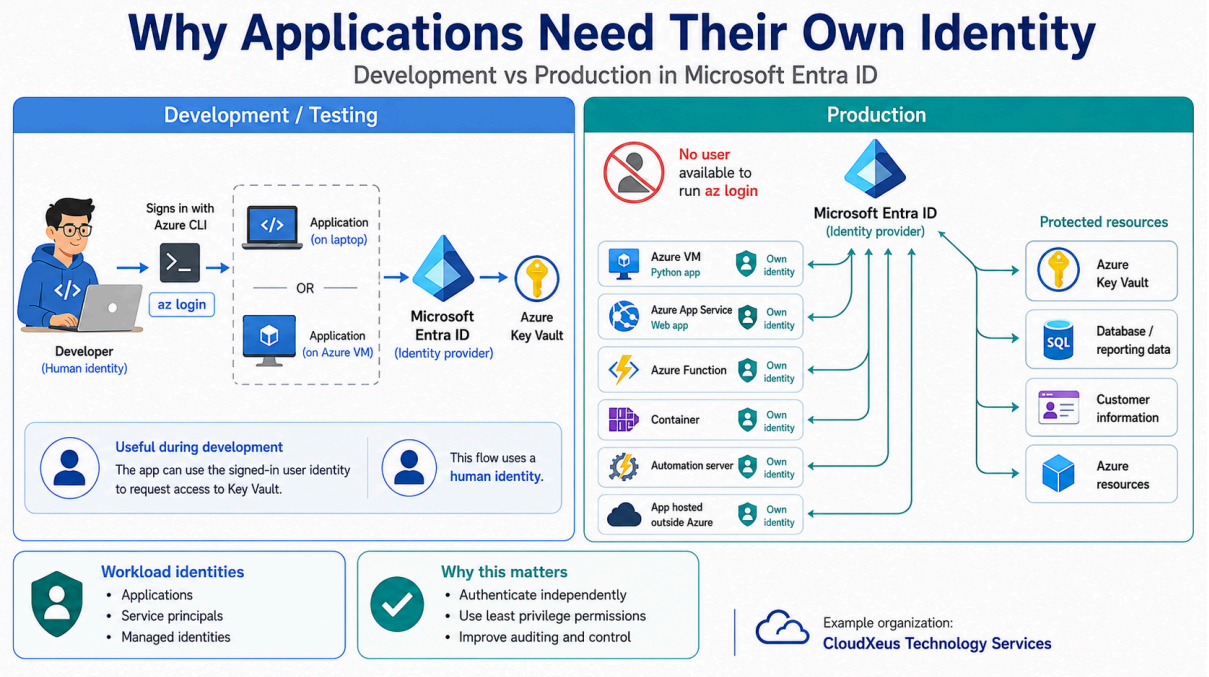
```
az login --use-device-code
```

Understanding Virtual Network Service Endpoints


Azure Key Vault Access from an Azure Virtual Machine using a Virtual Network Service Endpoint
Conceptual Azure architecture
Virtual network
vnet-northeurope-1
Region: North Europe
Subnet
snet-web-dev-neu
10.0.0.0/24
Service endpoint enabled
Microsoft.KeyVault
Virtual machine
vm-web-dev-01
Ubuntu 24.04
Private IP: 10.0.0.4
Public IP: 20.166.93.250
Python app reads secret: dbsecret
2 Traffic originates from subnet snet-web-dev-neu
Azure backbone network
3 Microsoft.KeyVault service endpoint identifies the subnet
4 Key Vault firewall allows the approved subnet
Service endpoint traffic
Azure Key Vault
kv-security-dev-eus-01
Secret: dbsecret
Key Vault firewall / network rules
Allow public access from specific virtual networks and IP addresses
Allowed virtual network rule:
vnet-northeurope-1 / snet-web-dev-neu
5 Key Vault returns dbsecret
1 Python code runs on vm-web-dev-01
2 Traffic originates from subnet snet-web-dev-neu
3 Microsoft.KeyVault service endpoint identifies the subnet
4 Key Vault firewall allows the approved subnet
5 Key Vault returns dbsecret
Important concept
Service endpoints are enabled on the subnet.
The Key Vault still uses its public endpoint.
The firewall must explicitly allow the virtual network and subnet.

💡 A **virtual network service endpoint** allows resources inside an Azure subnet to securely access supported Azure services while keeping the traffic on the Microsoft Azure backbone. In the example, the virtual machine `vm-web-dev-01` runs inside the subnet `snet-web-dev-neu` and needs to retrieve the `dbsecret` secret from Azure Key Vault.
To enable this design, the **Microsoft.KeyVault service endpoint** is enabled on the subnet. The Key Vault firewall is then configured to allow that specific virtual network and subnet. This means Key Vault can distinguish traffic originating from the approved subnet and reject requests coming from network locations that have not been permitted.
An important distinction is that a service endpoint does **not** give Key Vault a private IP address. Key Vault still uses its public service endpoint, but traffic from the approved subnet travels across the Azure backbone and is recognized as originating from that virtual network. A **private endpoint**, by comparison, gives the service a private IP address within the virtual network.
The key takeaway is that service endpoints provide an additional network security boundary, but they do not replace authentication or authorization. The application still needs a valid Microsoft Entra identity and the required Key Vault permissions. Think of the complete flow as: **approved subnet + authenticated identity + required permissions = successful Key Vault access**.

Why Applications Need an Identity in Microsoft Entra ID

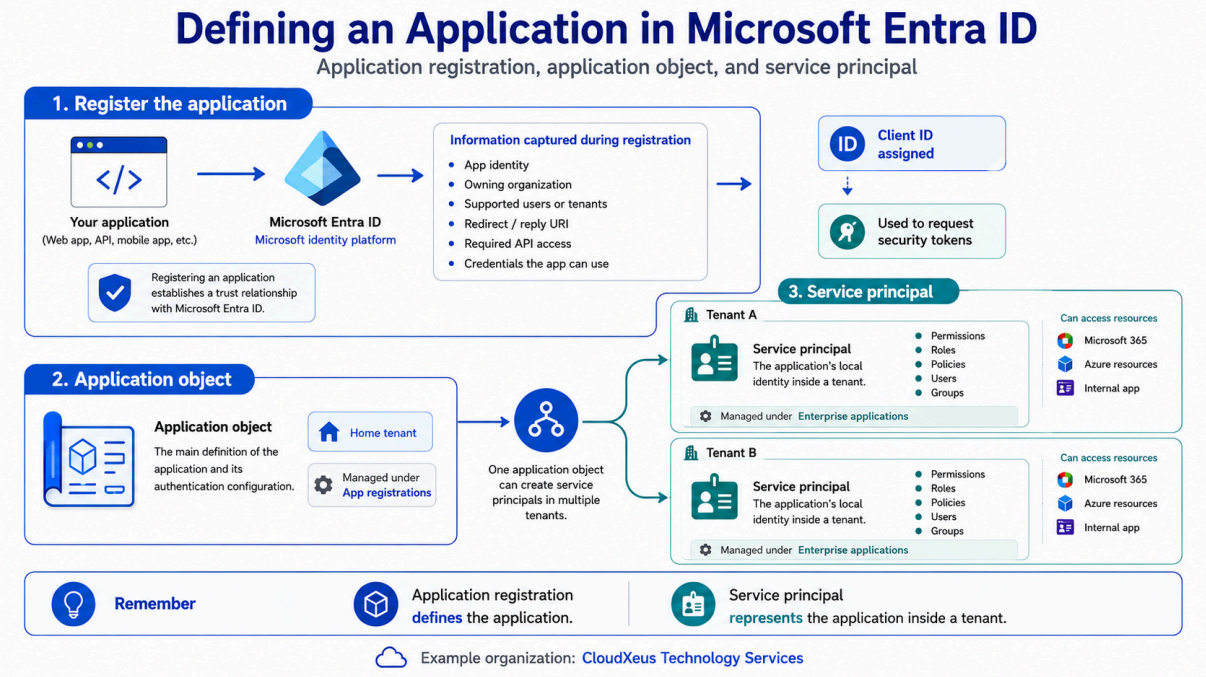

Why Applications Need Their Own Identity
Development vs Production in Microsoft Entra ID
Development / Testing
Signs in with Azure CLI
az login
Developer
(Human identity)
Application
(on laptop)
OR
Application
(on Azure VM)
Microsoft Entra ID
(Identity provider)
Azure Key Vault
Useful during development
The app can use the signed-in user identity to request access to Key Vault.
This flow uses a human identity.
Production
No user available to run az login
Microsoft Entra ID
(Identity provider)
Protected resources
Azure VM
Python app
Azure App Service
Web app
Azure Function
Container
Automation server
App hosted outside Azure
Own identity
Azure Key Vault
Database / reporting data
Customer information
Azure resources
Workload identities
Applications
Service principals
Managed identities
Why this matters
Authenticate independently
Use least privilege permissions
Improve auditing and control
Example organization:
CloudXeus Technology Services

During development, an application can often use the identity of the developer. For example, a developer can run `az login`, authenticate with Microsoft Entra ID, and allow a Python application to use that authenticated session when accessing resources such as Azure Key Vault. This works because a human user is available to complete the sign-in.

Production applications are different. An application running continuously on an Azure VM, App Service, Function, container, or automation server cannot depend on someone manually signing in. The workload therefore needs **its own identity** so that it can authenticate independently when no human user is present.

Microsoft Entra refers to identities used by applications and services as **workload identities**. These can include applications, service principals, and managed identities. Giving each workload its own identity also allows administrators to assign only the permissions that application requires and makes auditing much clearer because activity can be associated with the application rather than a developer's personal account.

The key takeaway is that **human users should authenticate using human identities, while production applications should authenticate using workload identities**. This allows applications to operate independently, supports least privilege, and avoids tying a production system to the lifecycle or permissions of an individual employee.



When an application needs to use Microsoft Entra ID for authentication, the application is first **registered**. During the registration process, Microsoft Entra captures information such as the application name, supported account types, redirect URIs, required API access, and authentication configuration. The application is also assigned an **Application (client) ID**, which uniquely identifies it.

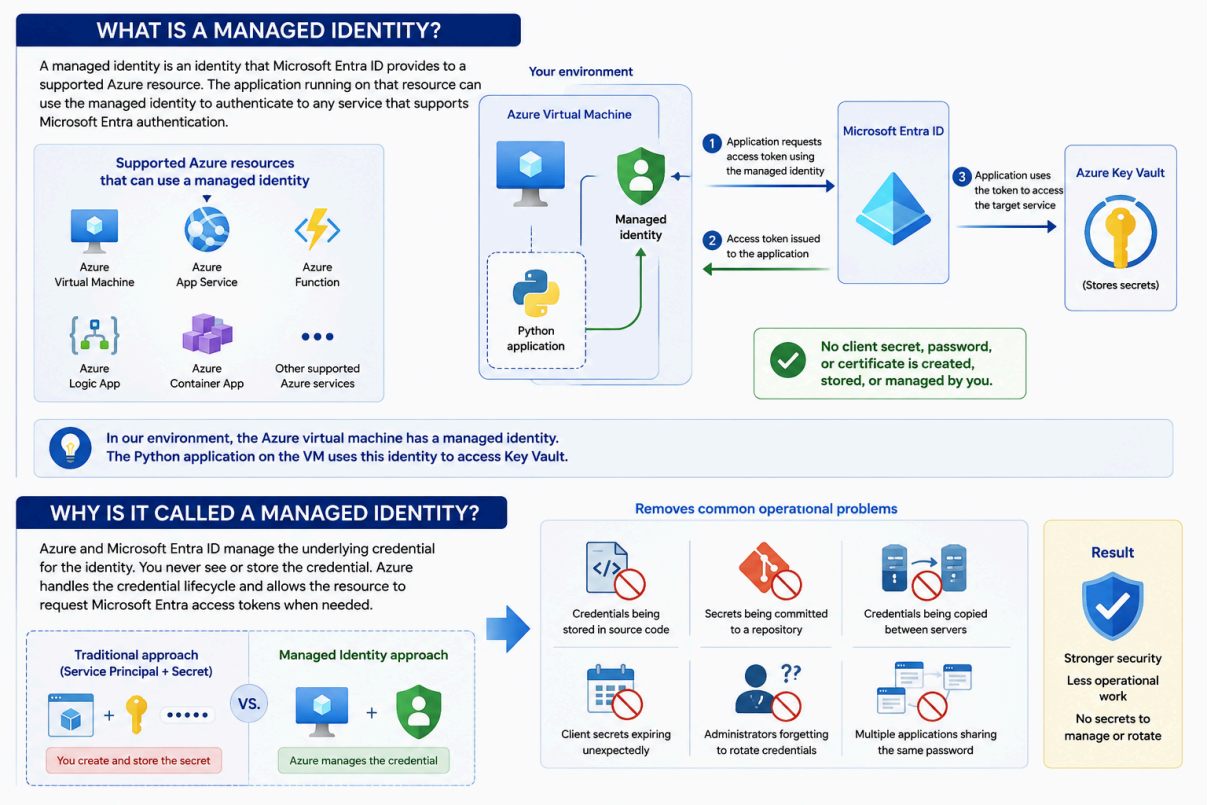
The registration creates an **application object** in the application's home tenant. The application object can be thought of as the main definition or blueprint of the application. It describes how the application integrates with Microsoft Entra ID and is primarily managed through the **App registrations** area.

A **service principal** represents that application inside a particular Microsoft Entra tenant. This is the tenant-local identity that can receive permissions, roles, policies, and access to resources. For a multitenant application, there can be one application object in the home tenant but separate service principals in every tenant where the application is used. Service principals are commonly managed through **Enterprise applications**.

The easiest relationship to remember is: **application registration is the process, the application object defines the application, and the service principal represents the application inside a tenant**. For exam scenarios, also remember: **App registrations = application**

definition, while **Enterprise applications = tenant-specific service principal**.

Managed Identities for Azure Resources



A **managed identity** gives a supported Azure resource its own Microsoft Entra identity. An application running on that resource can then authenticate to services such as Azure Key Vault without developers having to create or store a client secret, password, or certificate. Azure manages the underlying credential on behalf of the workload.

In the example, a Python application runs on an Azure virtual machine. When the application needs to access Key Vault, it requests an access token using the VM's managed identity. Microsoft Entra ID provides the token, and the application presents that token to Key Vault. Key Vault then checks whether the managed identity has permission to perform the requested operation.

Enabling a managed identity does **not** automatically provide access to Azure resources. The identity still needs authorization. For example, if

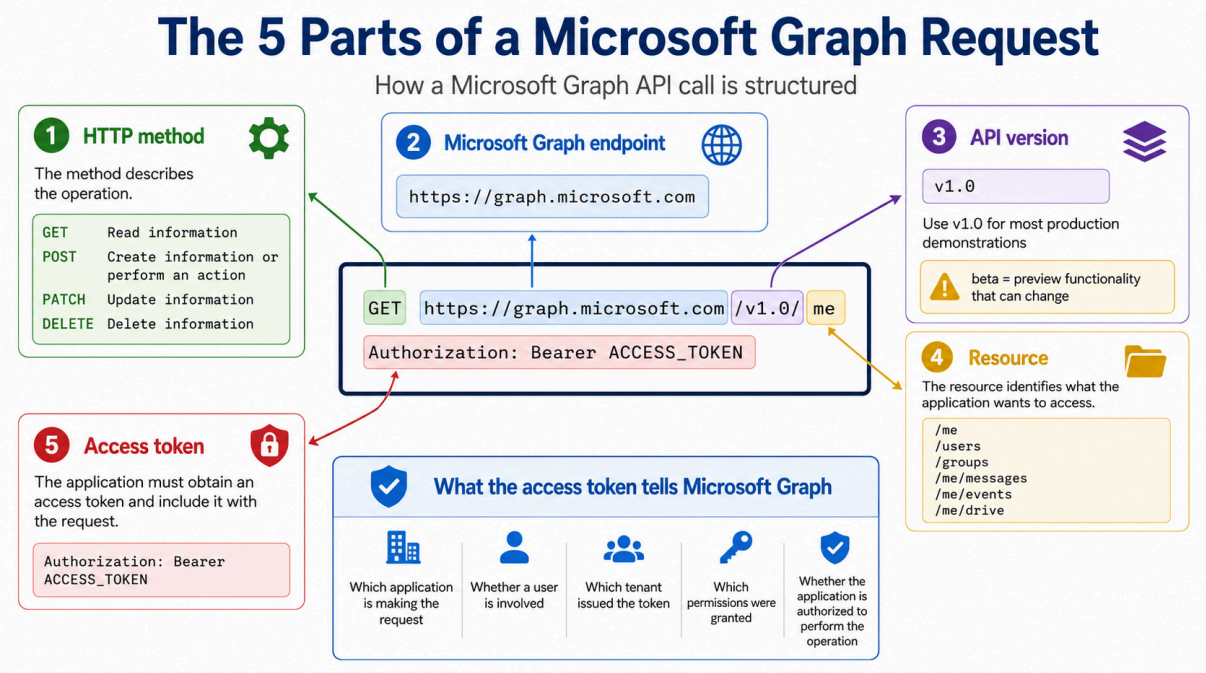
the application needs to read secrets, the VM's managed identity could be assigned the **Key Vault Secrets User** role at the appropriate scope. Managed identities solve the authentication and credential-management problem; Azure RBAC determines what the identity is allowed to do.

The major benefit is that developers no longer have application credentials to embed in source code, copy between servers, rotate manually, or accidentally commit to a repository. The key idea is: **give the Azure workload an identity, grant that identity least-privilege permissions, and let Azure manage the underlying credential.**

Introduction to Microsoft Graph API

https://learn.microsoft.com/en-us/graph/overview

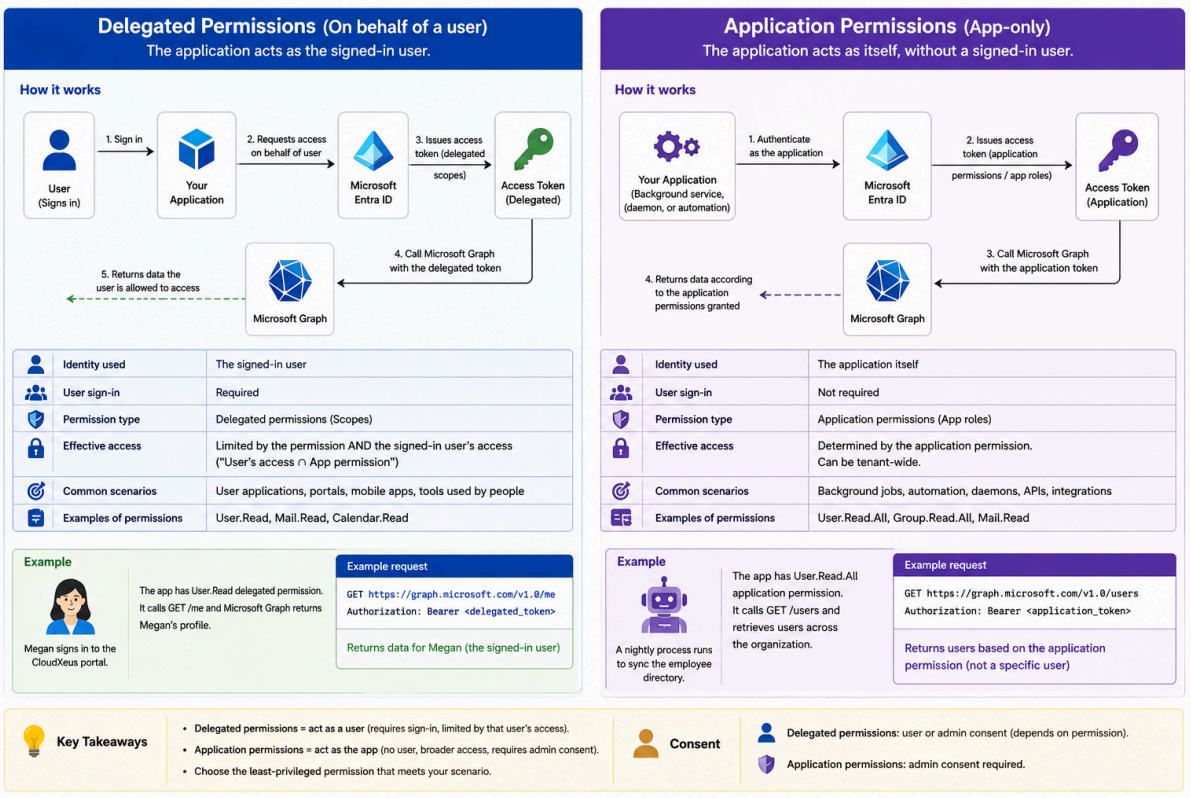
How a Microsoft Graph request works



A **Microsoft Graph request** is made up of five main parts: the **HTTP method**, the **Microsoft Graph endpoint**, the **API version**, the **resource**, and the **access token**. Together, these pieces define what operation the application wants to perform and whether Microsoft Graph should allow it. In the example shown, the request is a `GET`

request sent to `https://graph.microsoft.com/v1.0/me`, which means the application is trying to read information about the signed-in user.
The **HTTP method** tells Microsoft Graph what type of action to perform. For example, `GET` is used to read information, `POST` is used to create something or trigger an action, `PATCH` is used to update existing data, and `DELETE` is used to remove data. The **endpoint** is the base address for Microsoft Graph, and the **API version** determines which version of the API is being used. In most production-style demonstrations, `v1.0` is preferred, while `beta` is used for preview features that may change.
The **resource** identifies what the application wants to access. In the diagram, `/me` refers to the current signed-in user, but other examples include `/users`, `/groups`, `/me/messages`, and `/me/events`. This is the part of the request that points to the object or data the application wants to read or manage. So the full path effectively combines the endpoint, version, and resource into a usable Graph URL.
The final and critical component is the **access token**, which is included in the Authorization header as a bearer token. This token tells Microsoft Graph which application is making the request, whether a user is involved, which tenant issued the token, and what permissions have been granted. The key takeaway is that a Microsoft Graph call is not just a URL—it is a structured API request where the method, path, version, and token all work together to determine **what is being requested and whether the request is authorized**.

Delegated and application permissions



Microsoft Graph permissions generally fall into two important models: **delegated permissions** and **application permissions**. With delegated permissions, a user is signed in and the application acts **on behalf of that user**. The access token contains delegated scopes, and the application can only perform operations permitted by both the granted scope and the signed-in user's own access. This is commonly used for interactive applications such as portals, desktop apps, and mobile apps.

With **application permissions**, there is no signed-in user. The application authenticates as itself using its Microsoft Entra workload identity and receives an app-only access token. The permissions are based on application roles rather than delegated scopes. This model is commonly used for background services, automation jobs, daemons, and integrations that need to operate even when no user is present.

For example, a delegated application with `User.Read` can call Microsoft Graph `/me` to retrieve information about the current signed-in user. An unattended synchronization application might instead receive the `User.Read.All` application permission and call `/users` to retrieve users across the organization. Application permissions can therefore

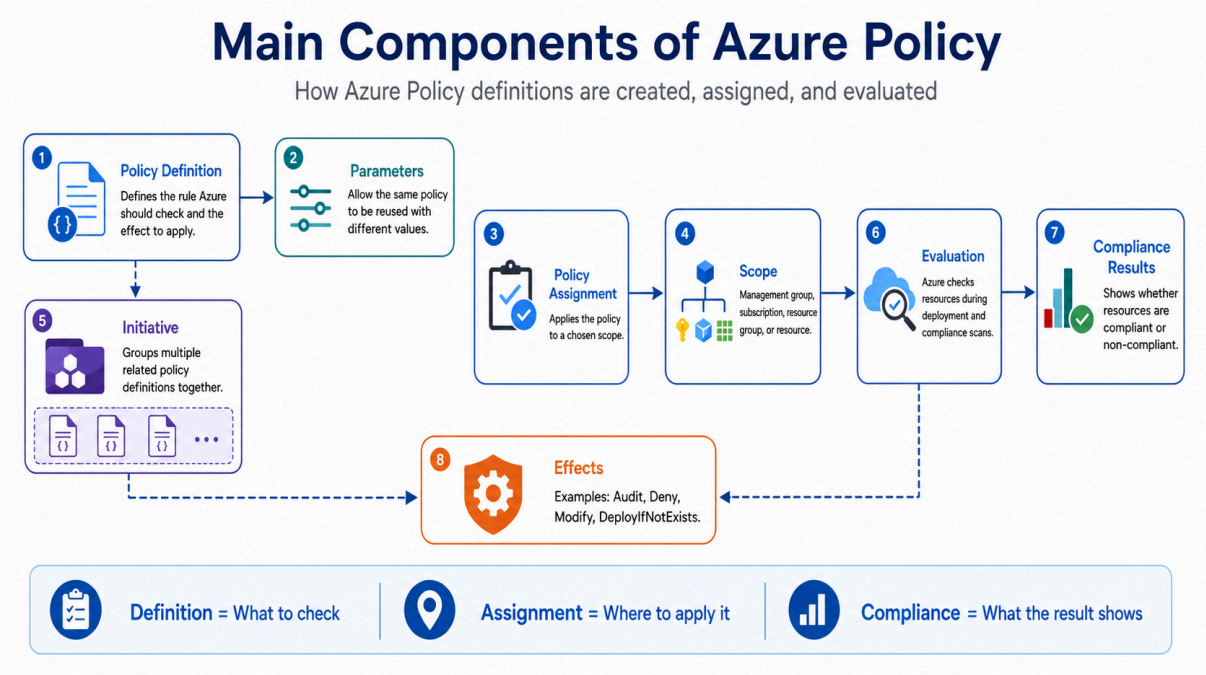
provide much broader access and normally require **administrator consent**, while delegated permissions may require user or administrator consent depending on the permission.
The key takeaway is to choose the model based on **who is performing the operation**. If the application is acting for a signed-in user, use delegated permissions and remember that access is constrained by both the application permission and the user's access. If the application must operate independently, use application permissions and grant only the minimum access required. Think: **Delegated = app acting as a user; Application = app acting as itself.**

Protecting Azure Resources with Resource Locks

https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/lock-resources

Enforcing Security Standards with Azure Policy

https://learn.microsoft.com/en-us/azure/governance/policy/overview



Azure Policy helps organizations enforce standards and assess compliance across Azure resources. The process starts with a **policy**

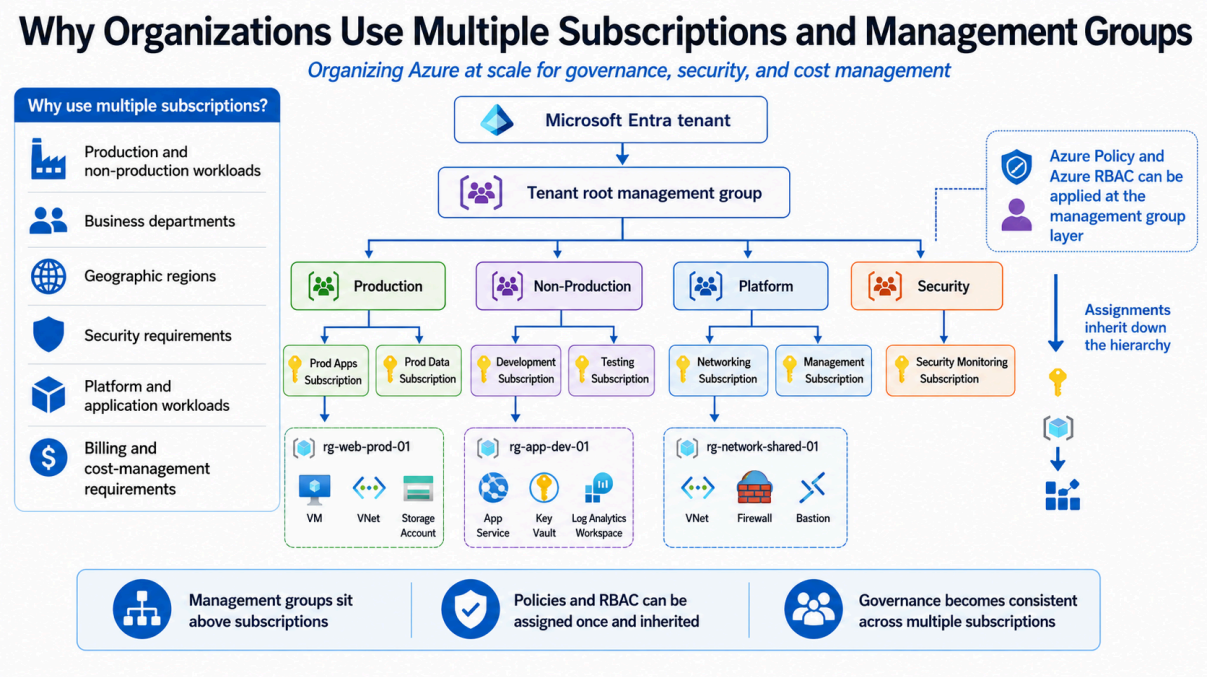
definition, which describes what Azure should evaluate and which effect should apply when a resource matches the rule. **Parameters** make definitions reusable by allowing values such as approved regions, required tags, or allowed resource types to be supplied when the policy is assigned.

A policy becomes effective when it is **assigned** to a scope. The scope determines where the policy applies and can include a management group, subscription, resource group, or individual resource. Multiple related policy definitions can also be grouped into an **initiative**, allowing administrators to assign and manage a collection of policies as one unit.

Azure then **evaluates** applicable resources against the assigned policy or initiative. The configured **effect** determines what happens when the policy condition is matched. Common examples include `Audit`, which reports non-compliance; `Deny`, which can block non-compliant resource operations; `Modify`, which can change supported resource properties; and `DeployIfNotExists`, which can deploy required configuration and support remediation of existing resources.

The result appears through Azure Policy **compliance information**, showing which resources meet the required standards and which do not. The simplest way to remember the architecture is: **Definition = what to check, Assignment = where to apply it, Effect = what should happen, Compliance = what the evaluation shows**. This makes Azure Policy a key governance tool for consistently enforcing security and configuration requirements at scale.

Understanding Azure Management Groups



Organizations often use **multiple Azure subscriptions** to separate workloads for operational, security, organizational, and cost-management purposes. For example, CloudXeus Technology Services might separate production from development and testing, place networking and management services into dedicated platform subscriptions, and use another subscription for centralized security monitoring. This provides clearer boundaries for ownership, billing, access, and resource organization as the Azure environment grows.

Management groups sit above subscriptions in the Azure hierarchy and allow organizations to manage several subscriptions as a logical unit. In the diagram, subscriptions are organized beneath Production, Non-Production, Platform, and Security management groups, all ultimately beneath the tenant root management group. A subscription can belong to one management group at a time, while management groups can contain other management groups or subscriptions.

The major security and governance benefit is **inheritance**. Azure Policy assignments and Azure RBAC role assignments applied at a management group can flow down to the subscriptions and resources beneath it. For example, CloudXeus could apply a security policy once at the Production management group instead of configuring the same policy independently on every production subscription. This

helps organizations enforce consistent governance as the number of subscriptions increases.

The key takeaway is to think of the hierarchy as **Management groups → Subscriptions → Resource groups → Resources**. Subscriptions provide administrative and organizational boundaries for different workloads, while management groups provide centralized governance across those subscriptions. For exam scenarios, remember: **management groups sit above subscriptions, and policies and RBAC assignments made higher in the hierarchy can be inherited by resources below them.**